Partie A — Manager l'innovation et sa dimension RSE (C2)

Q31 — Bilan des coûts réels

Pour établir ce bilan à partir du portail Azure (via *Cost Management* + *Billing*), l'analyse se fait généralement sur les composants clés consommés lors du projet :

1. **Les trois postes les plus élevés :**
    - **1. Les services de stockage (ADLS Gen2 / Stockage Blob) :** Pour l'hébergement des données brutes et structurées (zones Bronze/Silver/Gold) issues de vos extractions MongoDB.
    - **2. Les services de streaming / ingestion (Azure Event Hubs) :** Pour la capture en temps réel des événements (commandes, actions clients).
    - **3. Les instances de calcul / traitement (Azure Databricks) :** Pour faire tourner les pipelines de transformation, traiter les flux ou manipuler l'ontologie.

Q32 — Plus-value du streaming

Le traitement en temps réel via un pipeline de streaming (ex: Azure Event Hubs) procure des avantages compétitifs majeurs par rapport au batch (traitement par lots périodique) :

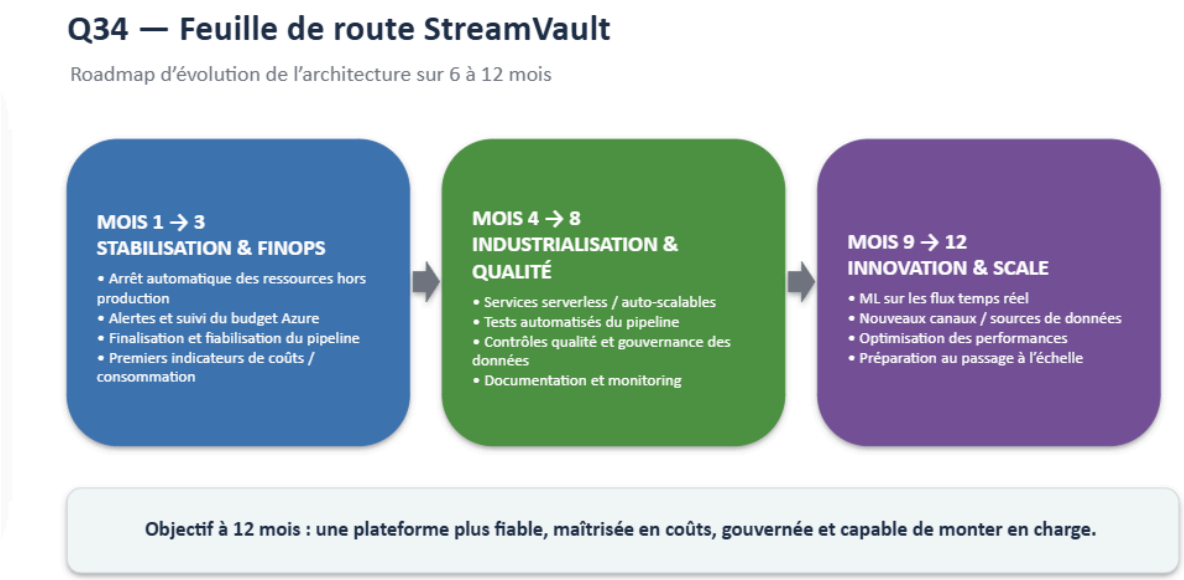
- Réactivité et personnalisation immédiate : Permet d'ajuster les recommandations de films ou livres à l'utilisateur *pendant* sa session de visionnage/lecture, et non le lendemain.
- Détection proactive et alertes instantanées : Identification immédiate d'anomalies (pics de charge anormaux, erreurs de commande, fraudes) pour déclencher des actions correctives en temps réel.
- Ce que le batch ne permet pas : Le batch est par définition rétrospectif (il traite ce qui s'est déjà passé, souvent la nuit). Il est incapable de répondre aux cas d'usage où la fraîcheur de la donnée se mesure en secondes (ex: gestion dynamique des stocks ou des flux clients en direct).

Q33 — Empreinte et sobriété

Comparatif (24/7 vs Arrêt automatique) : Laisser un cluster de calcul ou des environnements de test allumés 24h/24 et 7j/7 consomme inutileculement de l'énergie et des ressources cloud. Mettre en place un arrêt automatique en dehors des heures de bureau ou des pics d'activité permet de réduire le temps de fonctionnement d'environ 65% (soit un passage de 168h/semaine à environ 60h/semaine), réduisant d'autant la facture financière et l'empreinte carbone associée.

Lien avec l'ODD 9 (Industrie, innovation et infrastructure) : L'ODD 9 vise à bâtir une infrastructure résiliente, à promouvoir une industrialisation durable qui profite à tous et à encourager l'innovation. En intégrant l'écoconception logicielle et l'optimisation des ressources cloud (FinOps / GreenOps), StreamVault s'inscrit dans cette démarche d'infrastructures durables et économes en énergie.

Q34 — Feuille de route (Roadmap 6-12 mois)



<u>Partie B — Analyser le SI construit (C4)</u>

Q35 — Document de Conception Technique (DCT)

Projet StreamVault — Architecture de Données Hybride (Batch & Streaming)

Sommaire

1. Introduction et Objectifs
2. Modélisation des Données Opérationnelles (MongoDB)
3. Architecture Globale du Système
4. Description Détaillée des Composants Azure
5. Flux de Données et Formats Échangés
6. Choix Techniques et Sécurité

1. Introduction et Objectifs

Le projet **StreamVault** déploie une architecture cloud moderne sur Microsoft Azure pour centraliser, traiter et analyser des données multimédias et transactionnelles. Ce document formalise la conception technique du système, combinant l'ingestion de flux en temps réel (streaming) et l'intégration par lots (batch) à partir d'une base de données NoSQL (MongoDB).

2. Modélisation des Données Opérationnelles (MongoDB)

La base de données opérationnelle **streamvault** repose sur trois collections NoSQL distinctes :

- <u>Collection Customers</u> **:**
  - Contient les informations relatives aux clients de la plateforme.

- *Structure d'un document :* _id (identifiant textuel comme "Martin_Sophie"), nom, prenom, commune, code_postal, et type: "client".

- *Collection Movies_Livres :*
  - Regroupe le catalogue des œuvres (films et livres).
  - *Structure d'un document :* _id , title, year, genre, cast, directors, note_imdb, et type: "film".
- Collection commandes_clients **:**
  - Enregistre l'historique et les flux de commandes.
  - *Structure d'un document :* _id,id_commande , id_client, id_media, datetime et prix.

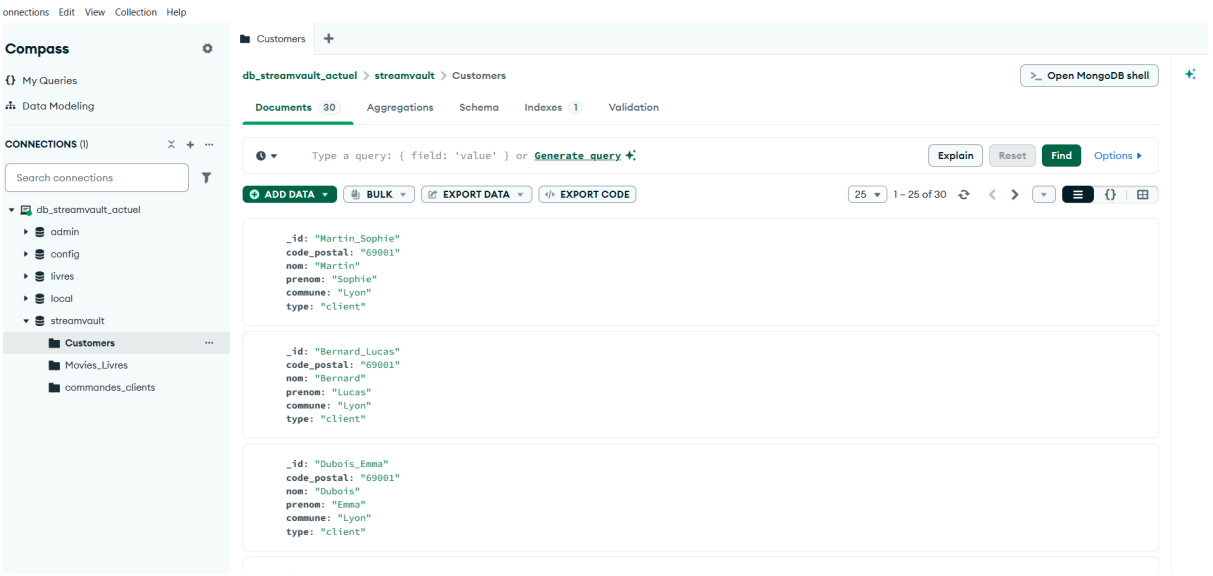


Figure: mongDb_Customers

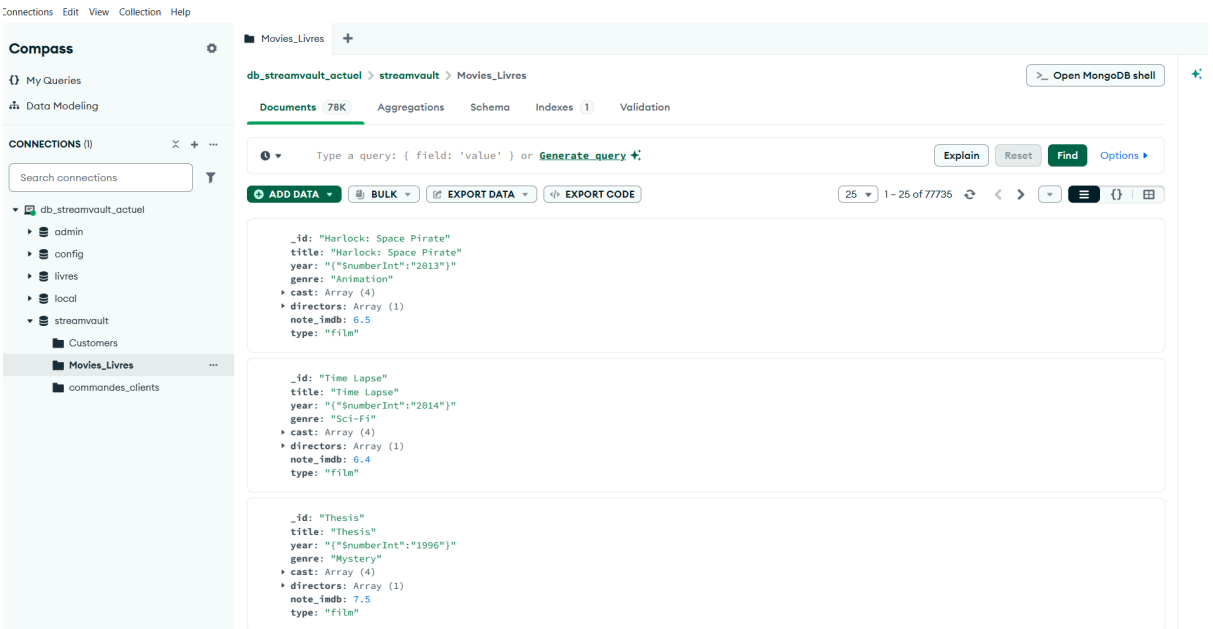


Figure: mongoDb_Movies_books

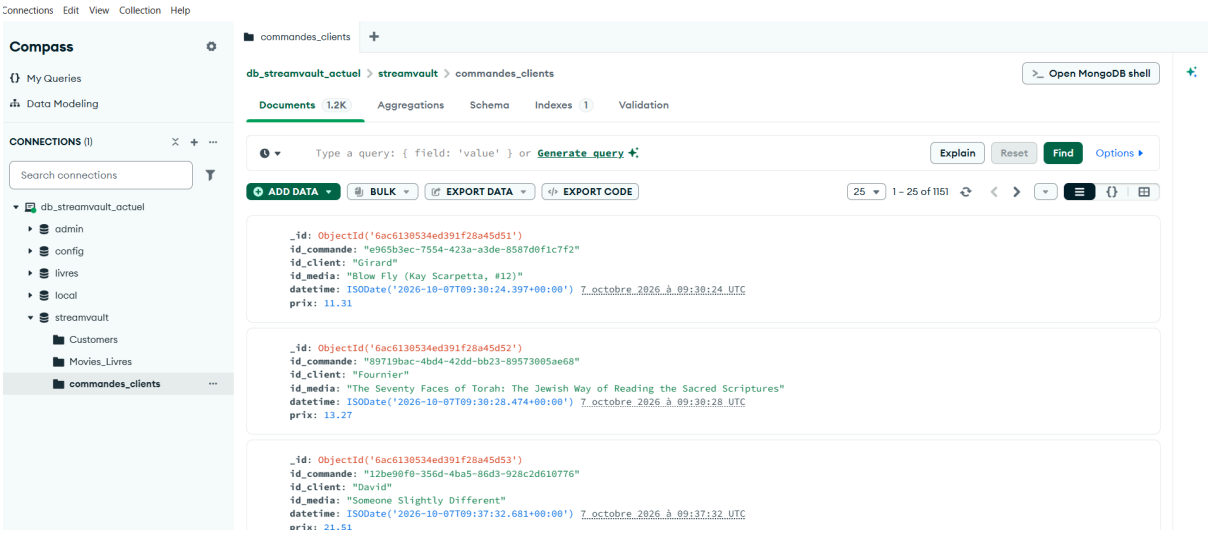


Figure: mongoDb_commandes_clients

3. Architecture Globale du Système

L'écosystème technique s'articule autour des briques suivantes :

- Source & Producteur **:** MongoDB couplé à un script Python (producteur.py) pour simuler l'injection de commandes en temps réel.
- Ingestion Temps Réel **:** Azure Event Hubs (commandes-hub) pour absorber le flux des commandes.
- Orchestration Batch **:** Azure Data Factory (ADF) pour piloter l'export et la synchronisation des catalogues (pipeline_Customers, pipeline_Movies_Books).
- Stockage (Data Lake) **:** Azure Data Lake Storage Gen2 (hamidestorage, conteneur clean) pour centraliser les fichiers JSON et les points de contrôle (*checkpoints*).
- Traitement Analytique **:** Azure Databricks pour exécuter les notebooks de transformation.
- Supervision **:** Azure Logic Apps (alerte-echec-commandes) pour la gestion proactive des alertes.

4. Description Détaillée des Composants Azure

4.1. Ingestion et Production de Flux

Le script producteur.py s'interface directement avec MongoDB à l'aide de requêtes aléatoires ($sample) pour piocher un client et un média. Il génère ensuite une charge utile JSON contenant le détail de la transaction et l'envoie en continu vers **Azure Event Hubs** via le protocole AMQP sur WebSockets.

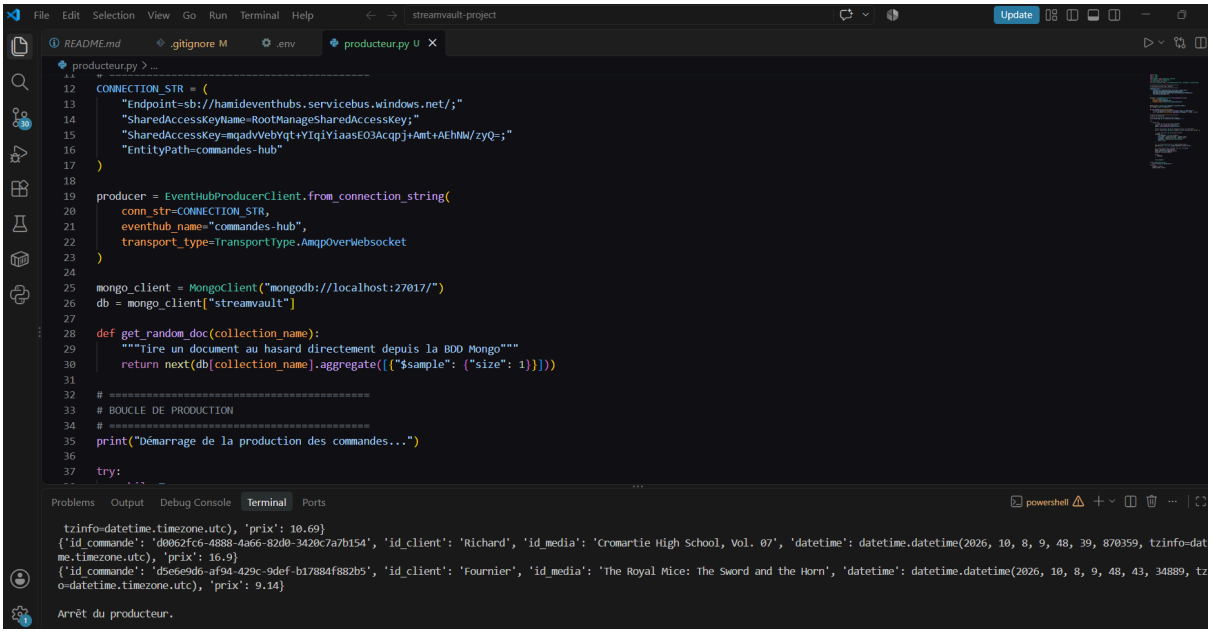


Figure: script_producteur.py

4.2. Orchestration (Azure Data Factory)

ADF gère le transfert des données statiques ou périodiques depuis MongoDB vers le Data Lake à travers des pipelines spécialisés (pipeline_Customers pour les clients, et pipeline_Movies_Books pour les films et livres), garantissant la mise à jour des référentiels.

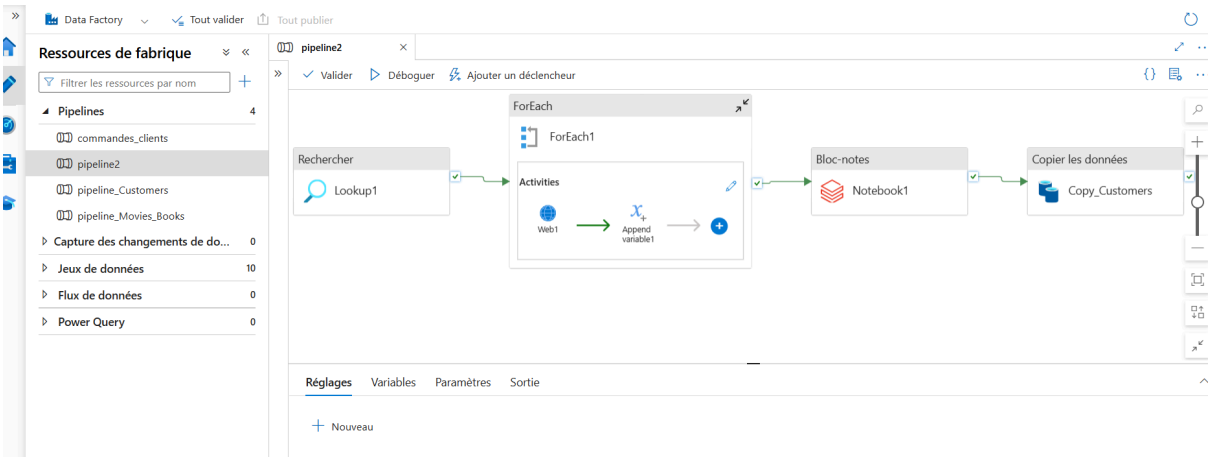


Figure:pipeline

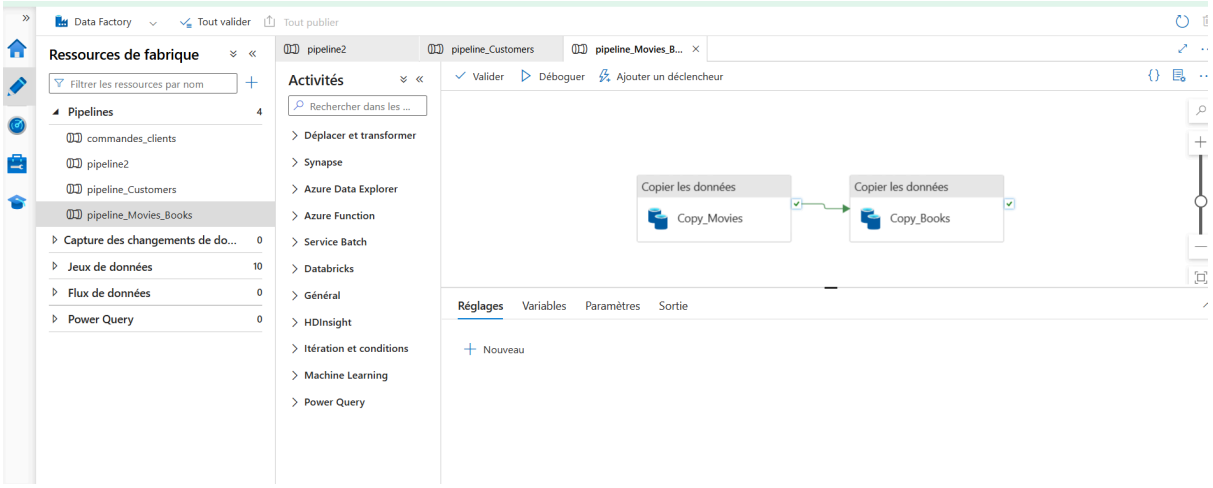


Figure: pipeline_movies_books

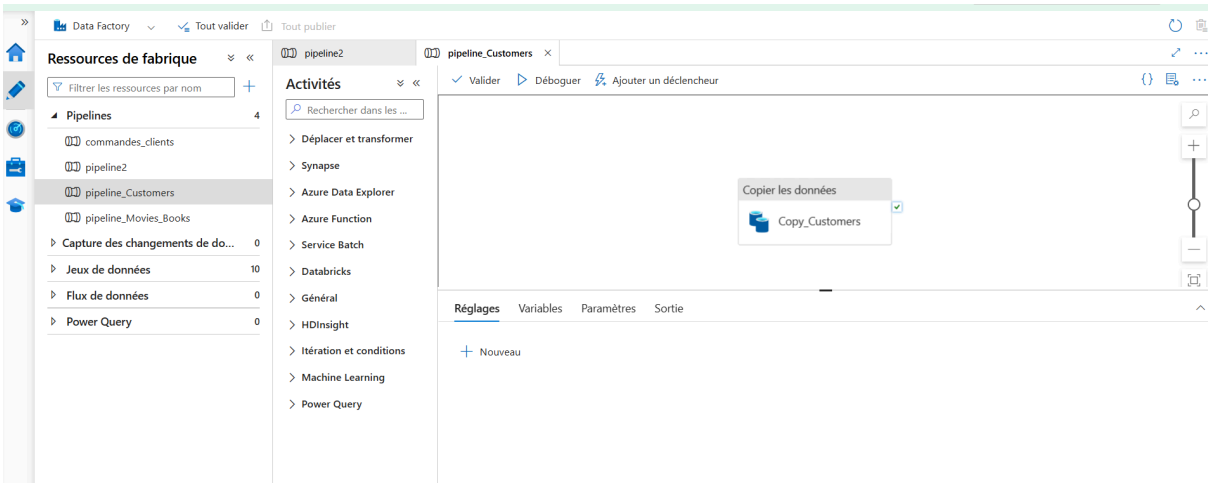


Figure:pipeline_customers

4.3. Stockage Persistant (ADLS Gen2)

Le conteneur clean du compte de stockage hamidestorage héberge les données au format brut ou structuré (books.json, clients.json, movies.json, ainsi que les répertoires de commandes et de checkpoints), permettant d'alimenter les couches successives (Bronze/Silver).

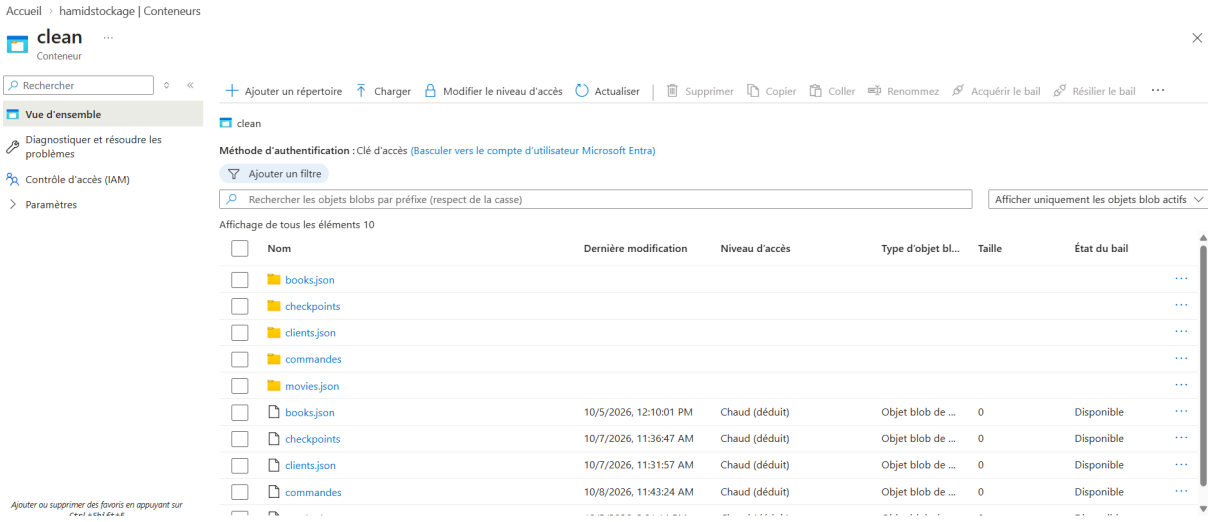


Figure:conteneur_clean

5. Flux de Données et Formats Échangés

1. Extraction / Capture : Les données issues de MongoDB sont converties et émises au format JSON.
2. Transit : Les flux de commandes transitent par Azure Event Hubs tandis que les référentiels clients et médias sont synchronisés via Azure Data Factory.
3. Persistance : Stockage intermédiaire sous forme de fichiers .json dans le Data Lake Gen2.
4. Transformation & Sémantique : Les notebooks Azure Databricks nettoient et structurent ces flux pour les rendre interopérables, notamment en vue de leur alignement avec l'ontologie OWL construite sous Protégé.

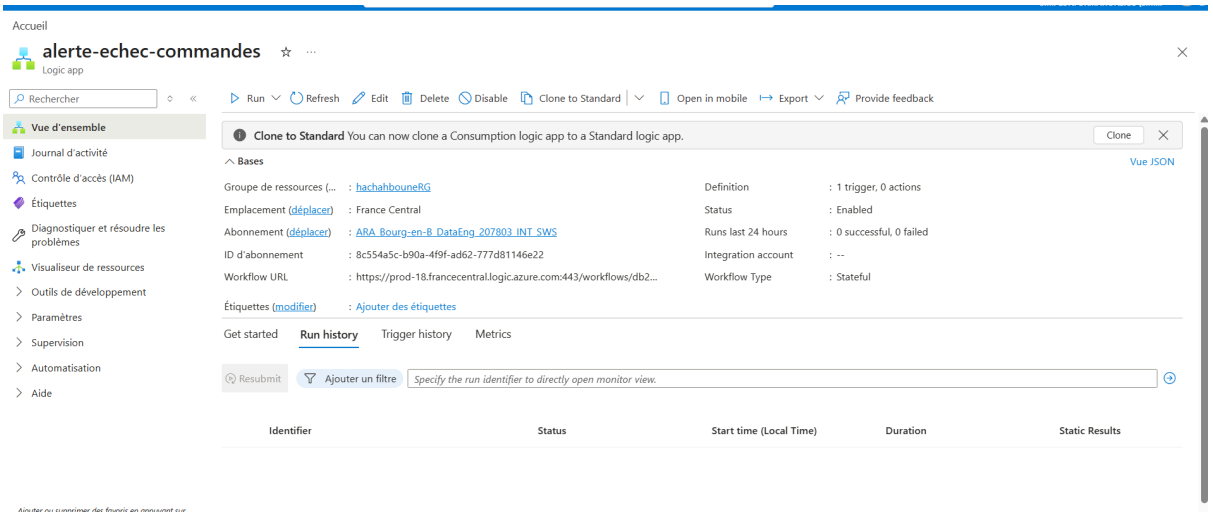
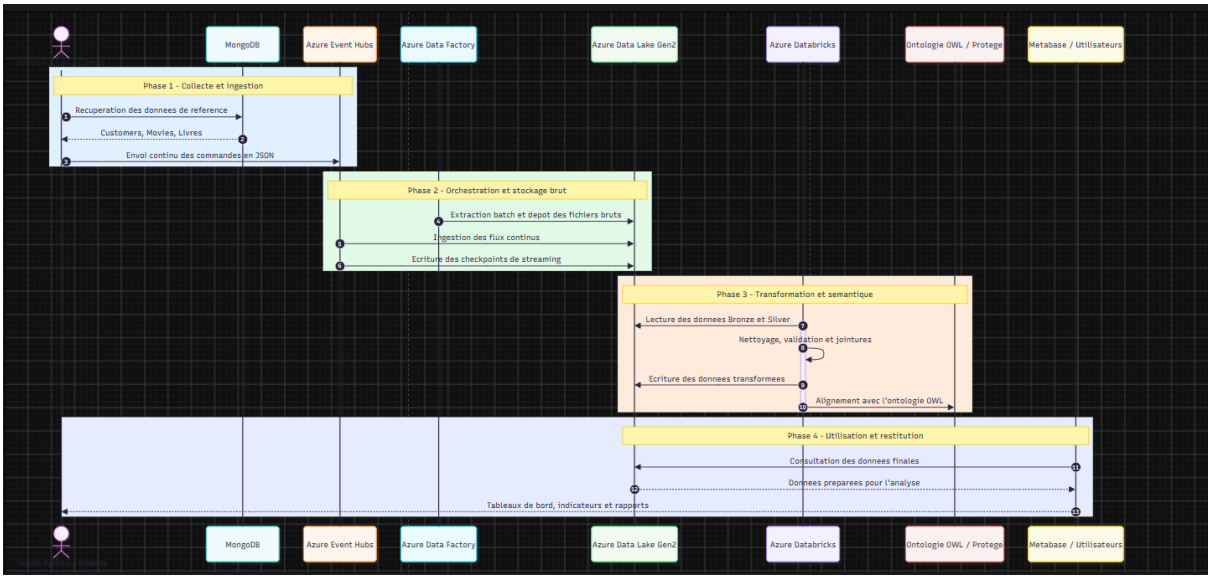


Figure: alerte-échec-commandes

6. Choix Techniques et Sécurité

- Hybridation Batch / Streaming **:** Séparation claire entre le traitement par lots pour les catalogues statiques (MongoDB → ADF → ADLS) et le temps réel pour les flux transactionnels (Script Python → Event Hubs).
- Sécurité et Traçabilité : Isolation des accès par clés de compte de stockage, chaînes de connexion sécurisées pour Event Hubs, et surveillance automatisée des erreurs via Azure Logic Apps.

Q36 — Modélisation formelle (Pipeline de données)



Explication du pipeline de données:

Phase 1 — Collecte et Ingestion Amont

- *Extraction des références (Messages 1 & 2) :* Le script Python amont interroge la base de données opérationnelle locale (**MongoDB**) pour récupérer les données de référence statiques nécessaires au contexte (profils clients, films, livres).
- *Flux de commandes temps réel (Message 3) :* Le script simule l'activité des utilisateurs en générant des commandes en continu et les pousse instantanément sous format JSON vers le bus de messages cloud (**Azure Event Hubs**).

Phase 2 — Orchestration et Stockage Brut

- *Ingestion Batch (Message 4) :* En parallèle, **Azure Data Factory** pilote l'extraction planifiée des données par lots (batch) et assure le dépôt des fichiers bruts dans les espaces de stockage correspondants du Data Lake.
- *Persistance du Streaming et Résilience (Messages 5 & 6) :* Les flux continus en provenance d'Event Hubs sont injectés dans **Azure Data Lake Gen2**. Simultanément, le système inscrit les points de contrôle (*checkpoints*) garantissant la tolérance aux pannes et le suivi des décalages (*offsets*).

Phase 3 — Transformation et Sémantique

- *Traitement Spark (Messages 7, 8 & 9) : Azure Databricks* lit les données issues des couches de stockage brut (Bronze/Silver), exécute les notebooks de nettoyage, de validation et de jointure, puis réécrit les données transformées.
- *Alignement Sémantique (Message 10) :* Les données structurées sont mises en correspondance avec l'ontologie du domaine à l'aide de l'outil **Protégé (Ontologie OWL)**, conférant une dimension sémantique forte au modèle.

Q37 — Analyse de l'intégration

L'architecture technique de StreamVault repose sur un modèle hybride combinant l'ingestion en temps réel (*streaming*) et l'intégration par lots (*batch*). L'interopérabilité entre les briques ***MongoDB, Azure Event Hubs, Azure Data Factory (ADF) et Azure Data Lake Storage Gen2 (ADLS Gen2*)** s'articule autour de flux standardisés garantissant le découplage et la résilience du système.

1. Analyse des flux et des formats de données

- *A. De MongoDB vers Azure Data Factory (Flux Batch Périodique)*
  - **Données transitant :** Données de référence statiques et semi-statiques de la plateforme, issues des collections `Customers` (profils clients) et `Movies_Livres` (catalogue des films et livres).
  - *Format d'échange :* Extraction initiale de documents BSON/NoSQL convertis en structures tabulaires ou fichiers JSON.
  - *Dépendances :* Dépendance directe d'ADF vis-à-vis de l'accessibilité de la base MongoDB (via chaîne de connexion sécurisée). Si MongoDB est indisponible, l'alimentation périodique du Data Lake est suspendue.
- *B. De MongoDB / Script Python vers Azure Event Hubs (Flux Temps Réel)*
  - *Données transitant :* Événements transactionnels continus générés dynamiquement par le script `producteur.py` (qui interroge MongoDB par tirage aléatoire pour associer un client à un média).
  - *Format d'échange :* Objets sérialisés en **JSON** (contenant l'identifiant de commande, du client, du média, la date et le prix) transitant via le protocole AMQP.
  - *Dépendances :* Le producteur Python nécessite la disponibilité simultanée de MongoDB (pour l'échantillonnage) et de l'espace de noms Azure Event Hubs (`commandes-hub`).
- *C. D'Event Hubs et d'ADF vers Azure Data Lake Storage Gen2 (Persistance)*
  - *Données transitant :* Les charges utiles de streaming en provenance d'Event Hubs et les copies de sauvegarde des référentiels orchestrées par ADF.
  - *Format d'échange :* Fichiers bruts au format **JSON** et dossiers de points de contrôle (*checkpoints*).
  - *Dépendances :* ADLS Gen2 (`hamidestorage`, conteneur `clean`) constitue le pivot central de stockage. Les composants d'aval (Azure Databricks)

dépendent de la présence et de l'intégrité de ces fichiers pour exécuter leurs traitements.

2. Synthèse des interdépendances techniques

Composant Aval	Composant Aval	Données Transmises	Format d'Échange	Type de Dépendance
MongoDB	Azure Data Factory	Référentiels (Clients, Catalogue)	JSON / Tabulaire	Synchrone / Périodique (Batch)
MongoDB + Python	Azure Event Hubs	Commandes en direct	JSON (Payloads)	Asynchrone / Continu (Streaming)
Event Hubs / ADF	ADLS Gen2	Fichiers bruts & *Checkpoints*	JSON / Fichiers plats	Persistant / Stockage central
ADLS Gen2	Azure Databricks	Données pour nettoyage & jointures	Parquet / JSON	Analytique (Aval direct)

3. Justification architecturale de l'intégration

- *Découplage opérationnel :* L'utilisation d'Azure Event Hubs comme zone tampon pour éviter de surcharger la base opérationnelle MongoDB lors de pics de trafic sur les commandes, garantissant ainsi la performance des applications transactionnelles.
- *Résilience et tolérance aux pannes :* La gestion des répertoires de *checkpoints* au sein d'ADLS Gen2 permet à Azure Databricks de reprendre les traitements de flux là où ils s'étaient interrompus en cas de coupure, évitant toute perte de données transactionnelles.

Partie C — Sécuriser un pipeline temps réel (C28)

Q38 — Test de continuité et de résilience du flux streaming

1. Protocole de test et interruption amont

- Objectif : Valider la capacité du pipeline à absorber une coupure brutale du producteur de données sans perte de messages (*zero data loss*).
- Manipulation : Arrêt volontaire et forcé du script d'injection (`producteur.py`) en cours d'exécution via le terminal.

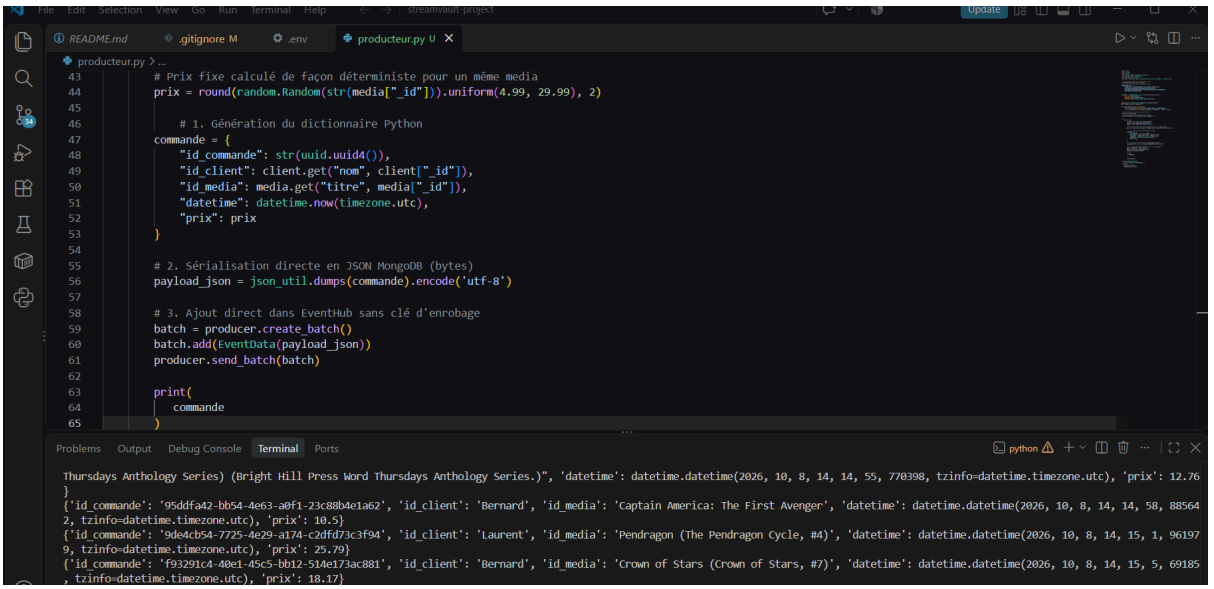


Figure 1 — Interruption volontaire et brutale du script producteur.

2. Traitement et reprise dans Azure Databricks

- *Comportement du consommateur :* Le notebook
- (basé sur Spark Structured Streaming) gère l'interruption sans erreur critique.
- *Sécurité des points de contrôle :* Grâce au paramètre de *checkpoint* configuré dans le Data Lake, le moteur de traitement est capable de conserver l'état de la lecture et de reprendre exactement au bon *offset* dès la reconnexion.

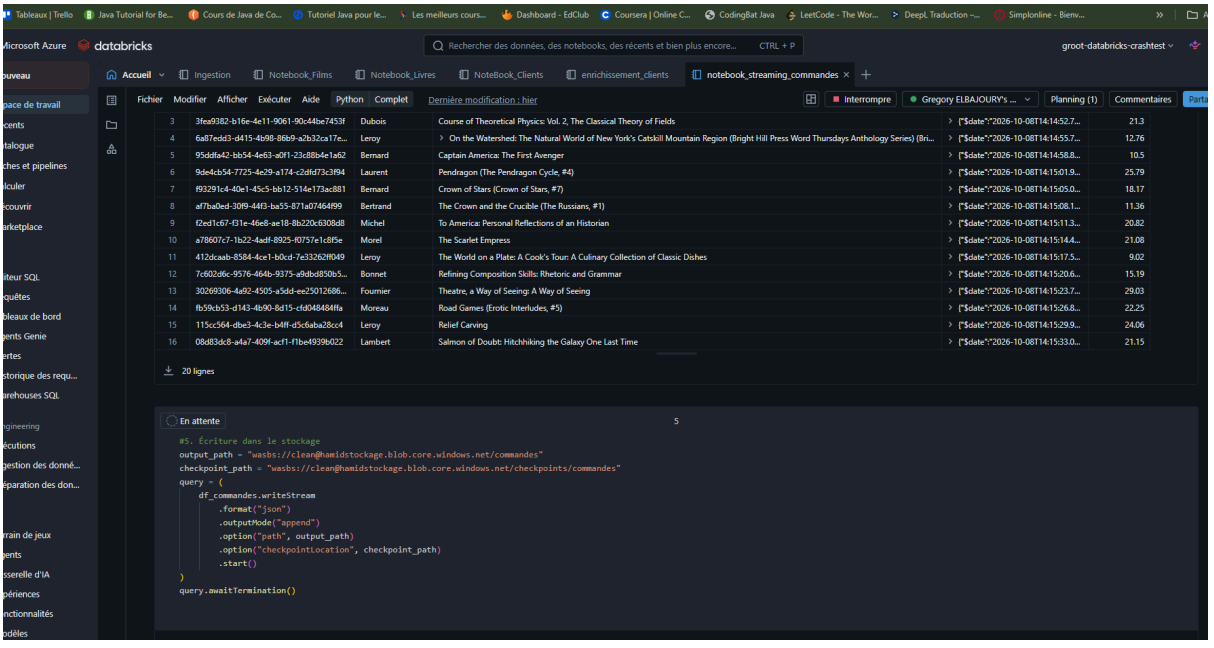


Figure 2 — Traitement et affichage des flux continus dans le notebook Databricks

3. Supervision et validation sur Azure Event Hubs

- *Analyse des métriques du bus :* L'observation du portail Azure sur l'espace de noms hamidEventHubs permet de suivre l'activité des requêtes et le volume de messages.

- *Vérification finale :* Le bus d'événements joue son rôle de tampon (*buffer*) en stockant les messages en attente. Les graphiques attestent de la reprise immédiate des flux au redémarrage.

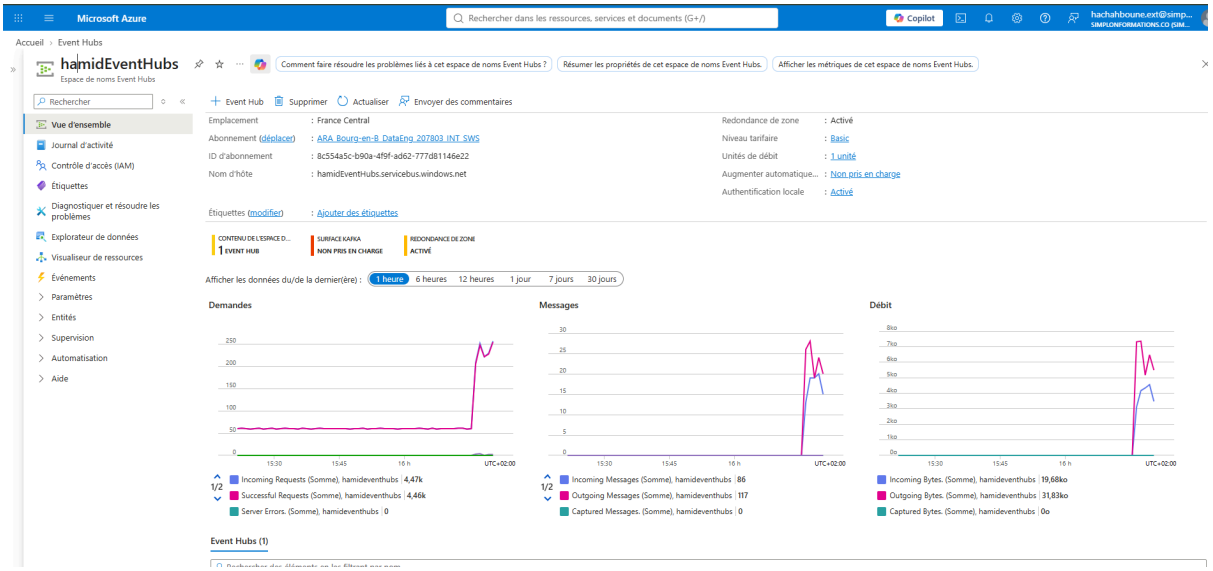


Figure 3 — Métriques d'activité et de trafic sur Azure Event Hubs.

4. Bilan du test

- *Résultat : Aucune commande n'a été perdue* lors de l'incident simulé.
- *Conclusion technique :* L'architecture démontre une excellente tolérance aux pannes, validant le modèle de traitement de type *at-least-once processing* propre aux architectures de streaming modernes sur Azure.

Q39 — Documenter la sécurisation du pipeline

Dans le cadre de l'implémentation et du prototypage de l'architecture StreamVault, les mécanismes de sécurité et de transport reposent sur les éléments suivants :

1. Gestion des accès et authentification

- **Connexions aux services cloud :** L'authentification auprès d'**Azure Event Hubs** et d'**Azure Data Lake Storage** s'appuie sur des clés d'accès configurées directement dans les paramètres de connexion des scripts et des notebooks Databricks.
- **Environnement opérationnel :** L'accès à la base de données opérationnelle s'effectue via un client local **MongoDB** dédié.

2. Chiffrement des données en transit

- **Protocoles de transport sécurisés :** L'ensemble des flux de données échangés entre le producteur local, le bus de streaming et les espaces de stockage cloud emprunte des canaux sécurisés et chiffrés (**TLS/HTTPS** et **AMQPS**).
- **Intégrité des charges utiles :** Ce chiffrement natif protège les structures de données (JSON) en mouvement contre toute tentative d'interception sur le réseau.

3. Piste d'évolution (Sécurisation cible)

- Bien que les clés soient actuellement intégrées dans les configurations des scripts pour les besoins du projet, l'industrialisation de cette architecture en production nécessiterait l'externalisation de ces secrets au sein d'un coffre-fort numérique dédié (type **Azure Key Vault**)

Q40 — Point de vulnérabilité et mesure de mitigation

1. Identification du point le plus critique

- **Le composant vulnérable :** Les chaînes de connexion et clés d'accès intégrées en clair (*hardcoded*) dans les scripts de production et les notebooks (visibles dans le code source ou les notebooks Databricks).
- **La ressource exposée :** L'espace de noms **Azure Event Hubs** (`hamidEventHubs`) et le compte de stockage cloud, en particulier l'utilisation de clés d'administration globales de type `RootManageSharedAccessKey`.

2. Risques et impacts en cas de compromission

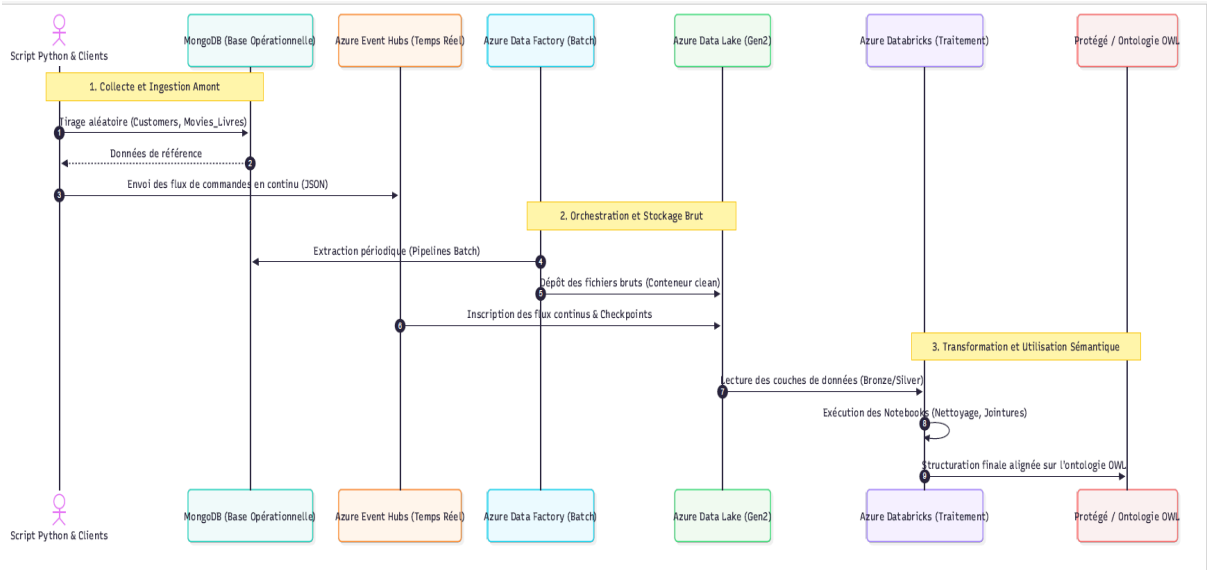
- **Contrôle total du bus de streaming :** Si un attaquant récupère ces identifiants (par exemple via un dépôt de code non sécurisé ou une fuite de configuration), il obtient des privilèges administratifs complets sur le bus d'événements.
- **Corruption ou injection de données :** Possibilité d'injecter de fausses charges utiles (*payloads* malveillants), de saturer les files d'attente, ou d'intercepter les données de commandes et profils clients en transit.

3. Mesure de mitigation réaliste (Plan d'action)

- **Application du principe du moindre privilège :** Remplacer la clé d'administration globale (*RootManageSharedAccessKey*) par des clés d'accès cloisonnées et spécifiques :
    - Une clé disposant uniquement des droits d'**Envoi (*Send*)** pour le script `producteur.py`.
    - Une clé disposant uniquement des droits d'**Écoute (*Listen*)** pour le notebook Databricks de consommation.
- **Sécurisation par coffre-fort numérique :** Migrer le stockage des secrets vers **Azure Key Vault** afin d'éviter toute exposition des clés en clair dans le code source, en autorisant uniquement les services légitimes à récupérer les jetons à l'exécution.

Partie D — Schéma final de votre propre architecture

Q41 — Dessiner votre architecture réelle



Phase 1 : Collecte et Ingestion Amont

- Le script Python interagit avec la base opérationnelle MongoDB pour le tirage aléatoire des données de référence (clients, films, livres) et pousse le flux de commandes en continu vers Azure Event Hubs.

Phase 2 : Orchestration et Stockage Brut

- Azure Data Factory pilote les extractions périodiques en mode batch, tandis que les flux continus et les points de contrôle (*checkpoints*) s'enregistrent dans Azure Data Lake Storage (Gen2).

Phase 3 : Transformation et Utilisation Sémantique

- Azure Databricks lit les données depuis les couches du Data Lake, exécute les notebooks de nettoyage et de jointure, pour enfin structurer les données finalisées en les alignant sur l'ontologie OWL (via Protégé).